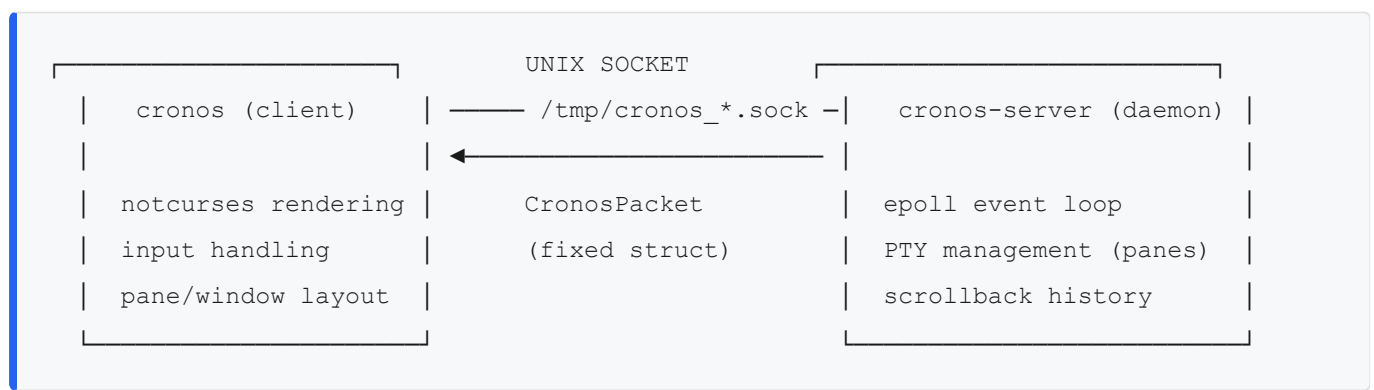


# Cronos - Terminal Multiplexer

Cronos is a `tmux`-style terminal multiplexer built on a Unix-domain-socket client/server architecture, with a `notcurses`-based terminal UI on the client and a PTY-managing daemon on the server. It supports persistent background sessions, pane splitting, multiple windows (tabs), scrollback, and a configurable keymap/color scheme.

## 1. Architecture

Cronos is split into two binaries that never share memory — all communication happens over a Unix domain socket using a fixed-size binary packet format.



### 1.1 The server ( `cronos-server` )

- Forks into the background via `daemon(0, 0)` immediately on launch - the process detaches from the controlling terminal and survives client disconnects.
- Owns one `ServerPane` struct per pane: a PTY master fd, the child shell's PID, and an active flag. Up to `MAX_PANES` (10) panes per session.
- Runs a single-threaded `epoll` loop multiplexing:
  - the Unix listening socket (accepts one client at a time),
    - the connected client socket (keystrokes + commands from the client),
    - every pane's PTY master fd (shell output).
- Spawns shells via `posix_openpt` → `grantpt` → `unlockpt` → `fork` → `execvp`, with the child calling `setsid()` and `TIOCSTTY` so it becomes its own session/process-group leader. This is what lets the server later kill an entire pane's process tree with `kill(-shell_pid, SIGTERM)` (negative PID = kill the whole process group) instead of just the top-level shell.
- Maintains a single ring buffer (`history_buf`, 64 KB) used **only for pane 0** (the session's original/root pane) so a reattaching client can replay recent output.
- Maintains `split_history[]`, a log of every successful split response packet, replayed to a newly-connected client so it can reconstruct pane layout on reattach.
- Reads shell config from `~/.config/cronos/cronos.conf` at startup (`load_server_config`): a custom shell path and arbitrary environment variables to inject into every spawned shell.

## 1.2 The client ( `cronos` )

The client is now split across several translation units, each owning one concern:

| File                      | Responsibility                                                                                                                                                                      |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>client.c</code>     | Entry point, CLI dispatch ( <code>new</code> / <code>attach</code> / <code>list</code> / <code>kill</code> / <code>killall</code> ), the notcurses/epoll event loop, config loading |
| <code>pane.c</code>       | Pane lifecycle: split (horizontal/vertical), close + neighbor absorption, focus navigation, resize, rendering pty output into the right <code>ncplane</code>                        |
| <code>window.c</code>     | Window (tab) lifecycle: create, switch, next/prev, rename, footer tab bar rendering                                                                                                 |
| <code>keymap.c</code>     | Keybinding table, default bindings, config-driven custom bindings, dispatch of resolved <code>Action</code> s                                                                       |
| <code>parser.c</code>     | ANSI/VT100 byte-by-byte state machine (CSI, OSC, SGR colors, cursor movement) plus scrollback capture                                                                               |
| <code>guide.c</code>      | <code>--help</code> / <code>--inst</code> CLI text, scrollback overlay rendering/enter/exit                                                                                         |
| <code>session_ui.c</code> | In-app session manager overlay: list/switch/create/rename sessions from inside a running client                                                                                     |
| <code>cronos.h</code>     | Shared structs/enums/constants used by every client TU and by the wire protocol                                                                                                     |

Each pane owns its own `ncplane` (and optionally a `border_plane` for split dividers and a `scroll_overlay` plane when in scrollback mode). Panes are stored as a singly-linked list per `Window`; `ClientContext` tracks the currently active window and pane globally so input/rendering code doesn't need to thread pointers through every call.

## 2. Wire Protocol

All communication is a fixed-size, non-versioned binary struct sent whole over the socket with a blocking `write()` / `recv(..., MSG_WAITALL)` pair — there is no framing length prefix because the struct itself is fixed-size:

```
typedef struct {
    uint8_t type;           // PKT_TYPE_KEYSTROKE | PKT_TYPE_PTY_OUTPUT | PKT_TYPE
    _COMMAND
    int     pane_id;        // which server-side pane this applies to
    size_t  data_len;       // valid bytes in payload[]
    char    payload[BUFFER_SIZE]; // BUFFER_SIZE = 256
} CronosPacket;
```

Because `data_len` is advisory and `payload` is always the same fixed size, every send is exactly `sizeof(CronosPacket)` bytes regardless of how much real data it carries — simple, but it does mean a pane's single read/write chunk is capped at 256 bytes.

## 2.1 Packet types

| Constant                         | Value | Direction       | Meaning                                                |
|----------------------------------|-------|-----------------|--------------------------------------------------------|
| <code>PKT_TYPE_KEYSTROKE</code>  | 0     | client → server | Raw bytes to write into a pane's PTY master            |
| <code>PKT_TYPE_PTY_OUTPUT</code> | 1     | server → client | Raw bytes read from a pane's PTY master                |
| <code>PKT_TYPE_COMMAND</code>    | 2     | both directions | Control message; <code>payload[0]</code> is a sub-type |

## 2.2 Command sub-types ( `payload[0]` when `type == PKT_TYPE_COMMAND` )

| Constant                                          | Value | Sent by | Purpose                                                                                                                       |
|---------------------------------------------------|-------|---------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>REQ_SPLIT_VERT</code>                       | 1     | client  | Request a vertical split of the active pane                                                                                   |
| <code>REQ_SPLIT_HORZ</code>                       | 2     | client  | Request a horizontal split of the active pane                                                                                 |
| <code>RES_SPLIT_VERT_SU</code><br><code>CC</code> | 3     | server  | Split succeeded; <code>payload[1]</code> = new pane id                                                                        |
| <code>RES_SPLIT_HORZ_SU</code><br><code>CC</code> | 4     | server  | Split succeeded; <code>payload[1]</code> = new pane id                                                                        |
| <code>WINDOW_RESIZE</code>                        | 5     | client  | <code>payload[1]</code> = rows, <code>payload[2]</code> = cols for <code>pane_id</code> ; applied via <code>TIOCSWINSZ</code> |
| <code>RES_PANE_CLOSED</code>                      | 6     | server  | The PTY for <code>pane_id</code> exited; client should absorb/remove it                                                       |
| <code>REQ_CLOSE_PANE</code>                       | 7     | client  | Ask the server to <code>SIGTERM</code> a pane's process group                                                                 |
| <code>REQ_NEW_WINDOW</code>                       | 8     | client  | Ask the server to spawn a fresh PTY for a new window/tab                                                                      |
| <code>RES_NEW_WINDOW_SU</code><br><code>CC</code> | 9     | server  | New window's pane is ready; <code>payload[1]</code> = new pane id                                                             |
| <code>REQ_RENAME_SESSIO</code><br><code>N</code>  | 10    | client  | <code>payload[1..]</code> = new session name (null-terminated)                                                                |

Splits and new-window creation are always server-initiated round trips: the client never allocates a pane id itself. This keeps `next_pane_id` (and thus pane numbering) authoritative on the server, which matters because the same id space is reused for keystroke/resize routing ( `panes[target_id]` ).

## 2.3 Session rename flow

Renaming is handled cooperatively across three places, which is worth spelling out since it's easy to assume the server does all of it:

1. `session_ui.c`'s `do_rename()` calls `rename(2)` **directly on the socket and pid files on disk** ( `/tmp/cronos_<old>.sock` → `/tmp/cronos_<new>.sock` , same for `.pid` ). This works safely even while the server has the socket open and a client is connected through it, because a Unix-domain socket's bind is to the underlying inode, not the path string — renaming the directory entry doesn't invalidate the existing file descriptor or break the live connection.

2. The client then sends `REQ_RENAME_SESSION` so the **daemon's own bookkeeping** (`global_socket_path` / `global_pid_path`), used only for cleanup on `SIGTERM` matches the new filenames — otherwise the daemon would try to `unlink()` paths that no longer exist when it eventually shuts down.
3. The client updates its own `ctx->session_name` used for footer display and for building the socket path on a future re-run of `cronos attach <name>`.

If you skip step 1 and only send the packet, the underlying files are never actually renamed on disk and a subsequent `cronos attach <new-name>` will fail because no socket exists at that path — the current implementation avoids that by doing the `rename(2)` client-side before notifying the server.

---

## 3. Data Model (client side)

```
ClientContext
├─ Window *window_list_head → Window → Window → ... (linked list of tabs)
│
│                               │
│                               └─ Pane *pane_list_head → Pane → Pane → ...
│                               └─ Pane *active_pane
├─ Window *active_window
└─ Pane *active_pane (kept in sync with active_window's active_pane via
                     ctx_sync_window() every time the window changes)
```

Each `Pane` carries:

- its own `TerminalState` (the ANSI parser's cursor/color/scrollback state — panes never share parser state, even within the same window),
- its own `ncplane` (and optional `border_plane` for the divider drawn on a split, and `scroll_overlay` created lazily when scrollback mode is entered),
- geometry (`x`, `y`, `width`, `height`) used both for notcurses layout and for neighbor-detection math when focusing/resizing/closing panes.

Splitting works by shrinking the parent pane's plane, creating a 1-row/1-col border plane, and creating a new plane in the freed space — geometry is recomputed with plain arithmetic (`parent->height / 2`, etc.), not delegated to notcurses' own layout system. Pane-close absorption (`res_pane_close`) walks the pane list looking for a pane that is exactly adjacent on one side and extends its geometry to fill the closed pane's space, transferring border-plane ownership rather than leaking or double-freeing it.

---

## 4. Terminal Emulation (`parser.c`)

`parse_ansi_byte()` is a small explicit state machine (`STATE_GROUND` → `STATE_ESCAPE` → `STATE_CS` / `I_ENTRY` / `STATE_CSI_PARAM`, plus `STATE_OSC` / `STATE_OSC_ESCAPE`) fed one byte at a time as PTY

output arrives.

### Supported:

- Standard + bright SGR colors (30–37, 40–47, 90–97, 100–107), reset (O), and default fg/bg (39/49).
- Cursor positioning: `H` (CUP), `A` / `B` / `C` / `D` (relative up/down/right/left), `G` (column absolute).
- Erase-in-display (`J`, modes 0/1/2/3) and erase-in-line (`K`, modes 0/1/2 — note modes 1 and 2 currently both erase the *entire* line rather than just left-of-cursor for mode 1; this is a simplification, not full VT100 semantics).
- `\r`, `\n` (with scrollback capture, see below), `\b`, `\t` (8-column tab stops).
- OSC sequences (`ESC ]`) are recognized and *discarded* up through their terminator (`BEL` or ``ESC ``) rather than crashing the parser or leaking into the visible buffer — this means window-title-setting escape sequences, for example, are silently ignored.

### Not supported / known gaps:

- No save/restore cursor (`ESC 7` / `ESC 8`), no scroll-region (`DECSTBM`), no alternate screen buffer mode (`?1049h/1`) — some full-screen TUI programs (vim, htop) may render imperfectly.
- Sub-parameters via `:` (used by some Kitty-protocol / SGR-with-colon sequences) are accepted syntactically (so the parser doesn't get stuck) but the sub-value is discarded rather than interpreted.
- 256-color and true-color SGR forms (`38;5;n` / `38;2;r;g;b`) are not implemented — only the fixed 16/8+8 color codes above are handled.

## 4.1 Scrollback

Scrollback is captured opportunistically: right before a `\n` would scroll the last visible row off the top of a pane's plane, the parser snapshots that row's on-screen characters (via `ncplane_at_yx`) into a per-pane ring buffer (`state.scrollback[SCROLLBACK_MAX_LINES]`, 2000 lines × 256 chars). This means scrollback is a byte-level *rendered-cell* capture, not the original stream — colors and attributes on scrolled-back lines are not preserved, only the visible glyphs.

Entering scrollback mode (`PgUp`) creates a full-size overlay plane on top of the pane and renders `scroll_offset` pages worth of history with a small status line; `PgDn` walks back down and destroys the overlay once you reach the live view.

---

## 5. Session Persistence & Reattach

- `dump_history_to_client()` replays the 64 KB history ring buffer **tagged as** `pane_id = 0` to any newly-connected client, so the primary pane's recent scrollback survives a detach/reattach cycle.
- `split_history[]` is replayed first, so a reattaching client receives the same `RES_SPLIT_*_SUCC` packets it would have gotten live, letting `pane.c` rebuild the same split layout before any PTY output packets are replayed on top of it.
- **Known limitation:** only pane 0's raw output is captured into history. If you split panes and then detach, non-zero panes come back with their pane geometry restored (from `split_history`) but with a *blank* pane — you only see new output produced after you reattach, not what scrolled by while

detached.

- Because the daemon calls `daemon(0,0)` and ignores `SIGPIPE`, a detached client (socket closed) doesn't affect the running shells; the server simply sets `client_sock = -1` and keeps servicing PTYs until a client reconnects.

---

## 6. Session Management

---

### 6.1 CLI (outside a running session)

```
cronos new <name>      # spawn a new background daemon and attach to it
cronos attach <name>   # attach to an already-running daemon
cronos list            # list all active session names (scans /tmp/cronos_*.sock)
cronos kill <name>     # SIGTERM a session's daemon by pid file, then unlink its files
cronos killall         # SIGTERM every active session found in /tmp
cronos --help, -h      # usage
cronos --inst, -i      # keybinding cheat-sheet
```

`new` refuses to run if a socket for that name already exists (tells you to `attach` instead). Both `new` and `attach` retry connecting to the socket for up to ~2 seconds ( $20 \times 100$  ms) to give the freshly-forked daemon time to bind and listen.

### 6.2 In-app Session Manager ( `Ctrl+A` )

Rather than only supporting session actions from the shell prompt, Cronos now has an in-TUI overlay ( `session_ui.c` ) reachable from inside any attached session:

- **Arrow keys** — move selection between all sessions currently found on disk.
- **Enter** — switch to the selected session: the current client cleanly tears down notcurses/raw mode and `execlp("cronos", "cronos", "attach", <name>, NULL)` s into the target session, replacing the current process rather than spawning a second one.
- **N** — prompt for a new session name inline, fork a fresh `cronos-server` for it, then `exec` into `attach` for that new session the same way.
- **R** — prompt for a new name and rename the *current* session (see §2.3 for exactly what that does under the hood).
- **Esc / q** — close the overlay and return to the session unchanged.

This is what closes the "switch between two already-running sessions" and "in-app session create/rename" gaps noted in earlier review passes — those actions no longer require detaching to a shell prompt.

---

## 7. Windows & Panes

---

- A **window** is a named collection of panes with its own independent pane tree (its own root pane, its own split layout). Windows correspond to what tmux calls "windows" (tabs), not to a top-level OS window.
  - A **pane** is a rectangular region running one shell; only panes within a window can be split relative to each other.
  - Switching windows ( `Ctrl+] / Ctrl+[` / via `window_next` / `window_prev` ) hides every pane belonging to the outgoing window by moving them far off-screen ( `hide_window()` , offset = screen height + 200) rather than destroying/recreating them, then moves the incoming window's panes back to their real coordinates ( `show_window()` ). This avoids re-running split layout math on every tab switch, at the cost of keeping every window's panes allocated in memory for the life of the session.
  - The footer bar ( `window_render_bar` ) renders a tab strip ( `[win1*] [win2]` , active tab marked with `*` ) alongside session name and active pane number.
  - **Known gap:** there is currently no window-close action — `ACTION_NEW_WINDOW` / `window_next` / `window_prev` / `window_rename` exist, but nothing frees a `Window` struct or its panes once created, and there's no corresponding server-side "kill this window's panes" request. Windows accumulate for the life of the client process.
- 

## 8. Keybindings & Configuration

---

Config file location (same file drives both the daemon and the client): `~/.config/cronos/cronos.conf`

### 8.1 Server-side directives

```
shell = /usr/bin/zsh
env EDITOR = nvim
env MY_VAR = some_value
```

- `shell = <path>` overrides the default `/bin/sh` used to spawn every pane.
- `env <KEY> = <value>` (repeatable) — exported into every spawned shell's environment in addition to whatever the daemon inherited. Up to `MAX_ENV_VARS` (32) entries.

## 8.2 Client-side directives

```
color_fg = 220,220,220
color_bg = 0,0,0
border_color = 100,100,100

bind ctrl+n = split_vert
bind alt+left = resize_left
```

- `color_fg` / `color_bg` / `border_color` take three comma-separated 0–255 RGB values.
- `bind <modifier>key = action` registers a custom binding on top of (not replacing) the built-in defaults below. Recognized modifiers: `ctrl+`, `alt+` (or no prefix for an unmodified key). `parse_key_name()` currently recognizes the literal words `left`, `right`, `up`, `down`, or any single printable character — it does not yet parse function keys, `pgup` / `pgdn`, or multi-character key names for custom bindings.
- Recognized action names for `bind`: `detach`, `split_vert`, `split_horz`, `focus_left`, `focus_right`, `focus_up`, `focus_down`, `close_pane`, `resize_left`, `resize_right`, `resize_up`, `resize_down`. (Window/session actions like `new_window` / `session_menu` are bound in code but not yet exposed to `parse_action_name()` for custom rebinding.)

## 8.3 Default keybindings

| Key                                         | Action                                                            |
|---------------------------------------------|-------------------------------------------------------------------|
| <code>Ctrl+Q</code>                         | Detach from session                                               |
| <code>Ctrl+V</code>                         | Split active pane vertically                                      |
| <code>Ctrl+B</code>                         | Split active pane horizontally                                    |
| <code>Ctrl+H</code> / <code>L</code>        | Focus pane left / right                                           |
| <code>Ctrl+J</code> / <code>K</code>        | Focus pane down / up                                              |
| <code>Ctrl+W</code>                         | Close active pane (not pane 0)                                    |
| <code>Alt+←→↑↓</code>                       | Resize active pane toward a neighbor                              |
| <code>PageUp</code> / <code>PageDown</code> | Enter/scroll/exit scrollbar overlay                               |
| <code>Ctrl+N</code>                         | Create a new window (tab)                                         |
| <code>Ctrl+R</code>                         | Rename current window (currently hardcoded to "renamed" — see §9) |
| <code>Ctrl+A</code>                         | Open the session manager overlay                                  |
| <code>Ctrl+]</code> / <code>Ctrl+[</code>   | Next / previous window                                            |

Note: pane 0 (the very first pane of the very first window) cannot be closed with `Ctrl+W` — the server only honors `REQ_CLOSE_PANE` for `target_id > 0`. Closing it would be equivalent to ending the whole



session, which is instead what happens naturally if its shell exits (the daemon sets `running = 0` and shuts down).

---

## 9. Build & Installation

---

```
make          # builds both `cronos` and `cronos-server` in the current directory
sudo make install # installs both to /usr/local/bin (override with PREFIX=...)
make clean
```

Requirements: a C compiler (`gcc`), `notcurses` / `notcurses-core` development headers and libraries. The client is compiled from multiple sources in one invocation:

```
client.c pane.c keymap.c guide.c parser.c window.c session_ui.c
```

`cronos-server` runs the daemon and is expected to be resolvable either via `PATH` (once installed) or via `./cronos-server` relative to the client's working directory when run from a source checkout — `cronos new <name>` execs `./cronos-server` directly, so run `cronos new` from inside the build directory unless you've `make install`ed both binaries onto `PATH`.

---

## 10. Known Issues Summary

---

For anyone triaging bugs, the concrete gaps identified while reviewing this revision:

1. **No window-close.** Windows can be created, renamed, and cycled through, but never destroyed — no keybinding or protocol message tears one down.
2. **Scrollback replay on reattach is pane-o-only.** Split panes lose their pre-detach history even though their layout is correctly restored.
3. **Rename window is hardcoded.** `ACTION_RENAME_WINDOW` currently calls `window_rename(ctx, "renamed")` with a literal string rather than prompting the user for a name (unlike session rename, which does prompt via `session_ui.c`).
4. **Custom-bind action set is incomplete.** `parse_action_name()` doesn't recognize `new_window`, `next_window`, `prev_window`, `rename_window`, or `session_menu`, so those can't be rebound from the config file even though they have `Action` enum values and default bindings.
5. **`parse_key_name()` is minimal.** Only arrow keys and single characters can be named in a config `bind` line — no `pgup` / `pgdn` / function-key support for custom bindings.
6. **ANSI coverage gaps** (see §4): no alternate-screen-buffer support, no save/restore-cursor, no scroll regions, no 256-color/truecolor SGR. Full-screen curses-style programs may not render perfectly.
7. **`REQ_RENAME_SESSION` payload has no explicit bounds check** on the server side beyond the packet's own fixed `payload[256]` size; a name close to that length combined with the `"/tmp/crono`

`s_%s.sock"` format could theoretically approach `socket_path[256]`'s capacity, though `snprintf` bounds the write itself so this is a truncation risk rather than an overflow.